[2CEIT401 – Operating Systems]

Practical: 7

AIM- Apply the concept of CPU Scheduling Algorithms.

Ganpat University
॥ विद्यया समाजोत्कर्षः ॥
U.V. Patel College of Engineering

Department of Computer Engineering

Aim: Apply the concept of CPU Scheduling Algorithms.

1. Write a program to implement FCFS CPU scheduling Algorithm.

Code:

```
#include <stdio.h>

int main() {
    // First Come First Serve (FCFS) Scheduling Algorithm
    int n, i;
    int bt[20], at[20], wt[20], tat[20], pid[20];
    int current_time, start_time;
    float avg_wt = 0, avg_tat = 0;
    printf("Enter number of processes: ");
    scanf("%d", &n); printf("Enter Burst Time:\n");
    for (i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);
        pid[i] = i + 1;
    }

    printf("Enter Arrival Time:\n");
    for (i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &at[i]);
    }

    for (i = 0; i < n - 1; i++) {
        int j;
        for (j = 0; j < n - i - 1; j++) {
            if (at[j] > at[j + 1]) {
                int temp;
                temp = at[j];
                at[j] = at[j + 1];
                at[j + 1] = temp;

                temp = bt[j];
                bt[j] = bt[j + 1];
                bt[j + 1] = temp;

                temp = pid[j];
                pid[j] = pid[j + 1];
                pid[j + 1] = temp;
            }
```

```
        }
    }

    current_time = 0;
    for (i = 0; i < n; i++) {
        start_time = (current_time > at[i]) ? current_time : at[i];
        wt[i] = start_time - at[i];
        tat[i] = wt[i] + bt[i];
        current_time = start_time + bt[i];
        avg_wt += wt[i];
        avg_tat += tat[i];
    }

    printf("\nProcess\tArrival Time\tBurst Time\tWaiting Time\tTurnaround Time\n");
    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t\t%d\t\t%d\t\t%d\n", pid[i], at[i], bt[i], wt[i], tat[i]);
    }
    printf("\nAverage Waiting Time = %.2f", avg_wt / n);
    printf("\nAverage Turnaround Time = %.2f", avg_tat / n);
    return 0;
}
```

Output:

```
[24012011142@linuxserv prac7]$ ./que_1
Enter number of processes: 4
Enter Burst Time:
P1: 5
P2: 4
P3: 3
P4: 5
Enter Arrival Time:
P1: 1
P2: 2
P3: 1
P4: 5

Process Arrival Time    Burst Time      Waiting Time    Turnaround Time
P1      1               5               0               5
P3      1               3               5               8
P2      2               4               7               11
P4      5               5               8               13

Average Waiting Time = 5.00
[24012011142@linuxserv prac7]$
```

2. Write a program to implement SJF CPU scheduling Algorithm.

Code:

```
#include <stdio.h>
int main() {
    int n, i;
    int bt[20], at[20], wt[20], tat[20], ct[20], pid[20], done[20];
    int completed = 0, current_time = 0;
    float avg_wt = 0, avg_tat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter Burst Time:\n");
    for (i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);
        pid[i] = i + 1;
        done[i] = 0;
    }

    printf("Enter Arrival Time:\n");
    for (i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &at[i]);
    }

    while (completed < n) {
        int idx = -1;
        for (i = 0; i < n; i++) {
            if (!done[i] && at[i] <= current_time) {
                if (idx == -1 || bt[i] < bt[idx] || (bt[i] == bt[idx] && at[i] < at[idx])) {
                    idx = i;
                }
            }
        }

        if (idx == -1) {
            current_time++;
            continue;
```

```
        }

        wt[idx] = current_time - at[idx];
        ct[idx] = current_time + bt[idx];
        tat[idx] = ct[idx] - at[idx];
        current_time = ct[idx];
        done[idx] = 1;
        completed++;
    }

    for (i = 0; i < n; i++) {
        avg_wt += wt[i];
        avg_tat += tat[i];
    }

    printf("\nProcess\tArrival Time\tBurst Time\tWaiting Time\tTurnaround Time\n");
    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t\t%d\t\t%d\t\t%d\n", pid[i], at[i], bt[i], wt[i], tat[i]);
    }

    printf("\nAverage Waiting Time = %.2f", avg_wt / n);
    printf("\nAverage Turnaround Time = %.2f\n", avg_tat / n);
    return 0;
}
```

Output:

```
[24012011142@linuxserv prac7]$ ./que_2
Enter number of processes: 4
Enter Burst Time:
P1: 5
P2: 4
P3: 3
P4: 2
Enter Arrival Time:
P1: 1
P2: 2
P3: 1
P4: 5

Process Arrival Time    Burst Time      Waiting Time    Turnaround Time
P1      1               5               9               14
P2      2               4               2               6
P3      1               3               0               3
P4      5               2               3               5

Average Waiting Time = 3.50
Average Turnaround Time = 7.00
[24012011142@linuxserv prac7]$
```